

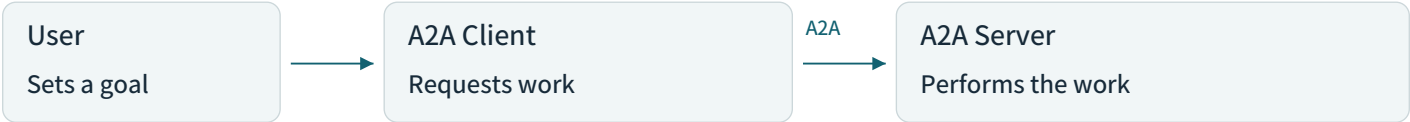
A2A at a glance

Agent2Agent Protocol / A map for understanding the big picture

Based on official v1.0 documentation · Checked 2026-10-07 · A4 / 1 page

A shared protocol for independent AI agents to delegate work and exchange progress and results.
Agents can collaborate without sharing their internal models, tools, or implementation.

1 Who does what?



The Client can be an app. The same agent can act as a Client when it contacts another agent.

2 The basic flow: discover → request → track → receive

- | | |
|-------------|--|
| 1. Discover | Read the Agent Card: skills, endpoints, and authentication requirements. |
| 2. Request | Send a Message (SendMessage). The response is a Message or a Task. |
| 3. Track | For a Task, retrieve its state (GetTask) or receive updates. |
| 4. Receive | Task outputs are Artifacts: reports, images, or structured JSON data. |

Example: a travel agent asks a hotel agent to find suitable accommodation and receives a shortlist.

3 Six terms to learn first

Agent Card	An agent’s business card: Skills (offered work) and supported features.
Message	One turn: a request, question, or reply. Carries the conversation.
Part	A content unit: text, file content or reference, or structured JSON.
Task	Trackable work with an ID, state, outputs, and message history.
Artifact	A concrete output of a Task, containing one or more Parts.
contextId / taskId	A group of related interactions / the ID of a specific Task.

4 Task states: running, waiting, or finished

Typical flow: SUBMITTED (accepted) → WORKING (in progress) → COMPLETED (done)

Waiting: INPUT_REQUIRED (more input) / AUTH_REQUIRED (more authentication) → then resume

Terminal: COMPLETED (done) / FAILED (error) / CANCELED (canceled) / REJECTED (declined)

A terminal Task cannot restart. Refinements create a new Task within the same contextId.

State names omit TASK_STATE_. This flow is an example; not every Task passes through every stage.

5 Receiving updates, and how MCP fits

Three ways to receive updates

Polling: the Client repeatedly checks the Task.
Streaming: receive progress and outputs live.
Push: receive key updates through a webhook.

A2A and MCP work together

A2A: ask an independent agent to do work.
MCP: an agent accesses tools and data.
Check the Card for Streaming / Push support.

In Rails: A2A is the external interface. Business logic, databases, and jobs stay in the app.

Bindings: JSON-RPC / gRPC / HTTP+JSON. Use standard auth mechanisms and app-level authorization.

Source: a2a-protocol.org/latest/ (section headings link to the official documentation)

Read next: Core Concepts → Life of a Task → Specification (Sections 3 and 4)